15
Bagging, forêts aléatoires et boosting

Automne 2026

Techniques d'apprentissage automatique — 420-C74-SF — gr. 10017
Spécialiste en solutions d'intelligence artificielle — M. Swawola, M.Sc.

NOUS ÉCLAIRONS.
VOUS BRILLEZ.

FORMATION CONTINUE
ET SERVICES AUX ENTREPRISES

Cégep de Sainte-Foy

Sommaire

1. Introduction au Bagging
2. Forêts aléatoires
3. Introduction au Boosting
4. AdaBoost
5. Gradient Boosting
6. Importance des variables explicatives
7. Lectures et références

Sommaire

1. Introduction au Bagging
2. Forêts aléatoires
3. Introduction au Boosting
4. AdaBoost
5. Gradient Boosting
6. Importance des variables explicatives
7. Lectures et références

Le Bagging est une **approche générale** applicable à de nombreux algorithmes d'apprentissage supervisé en régression et classification

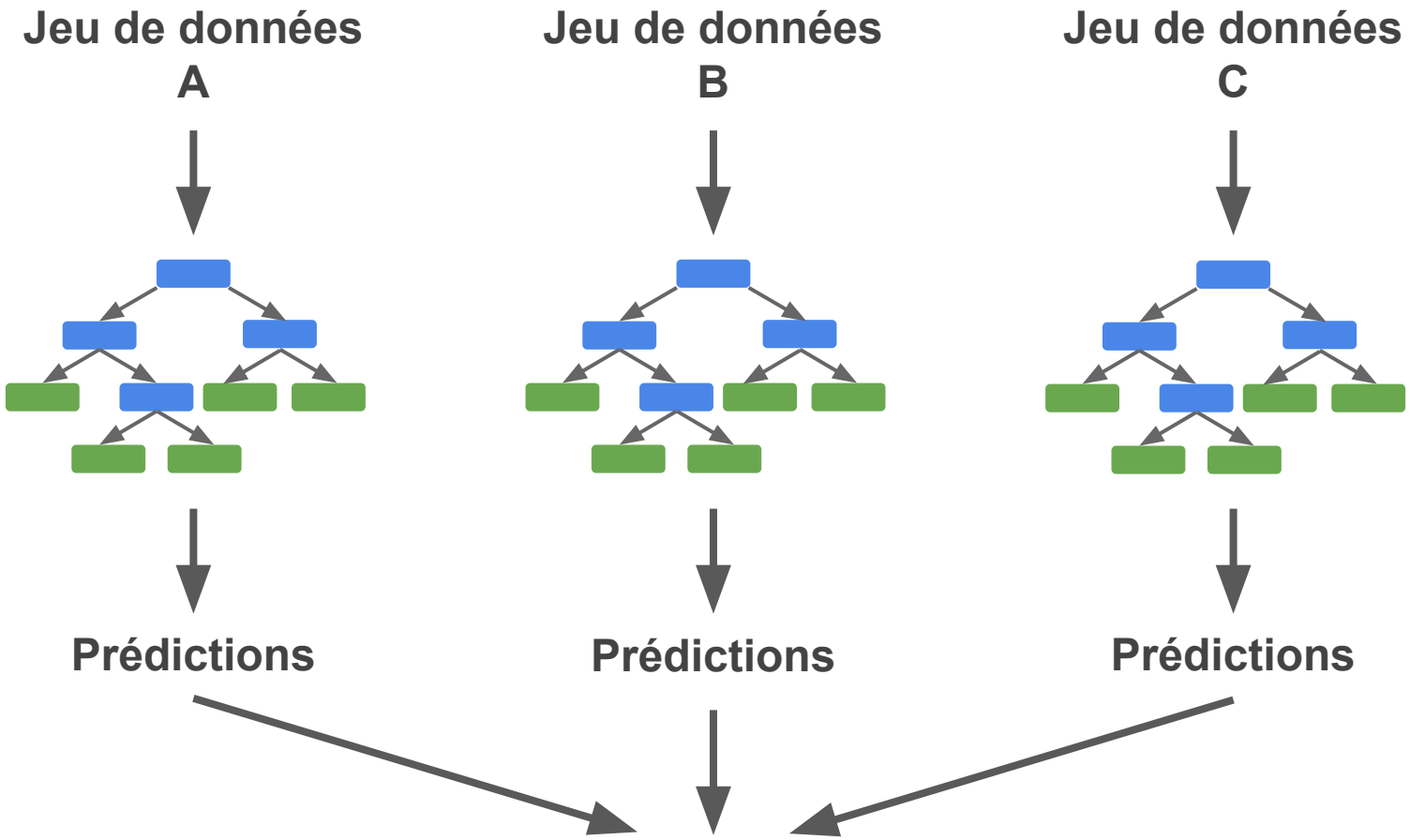
Nous allons voir ici le Bagging appliqué aux arbres de décision

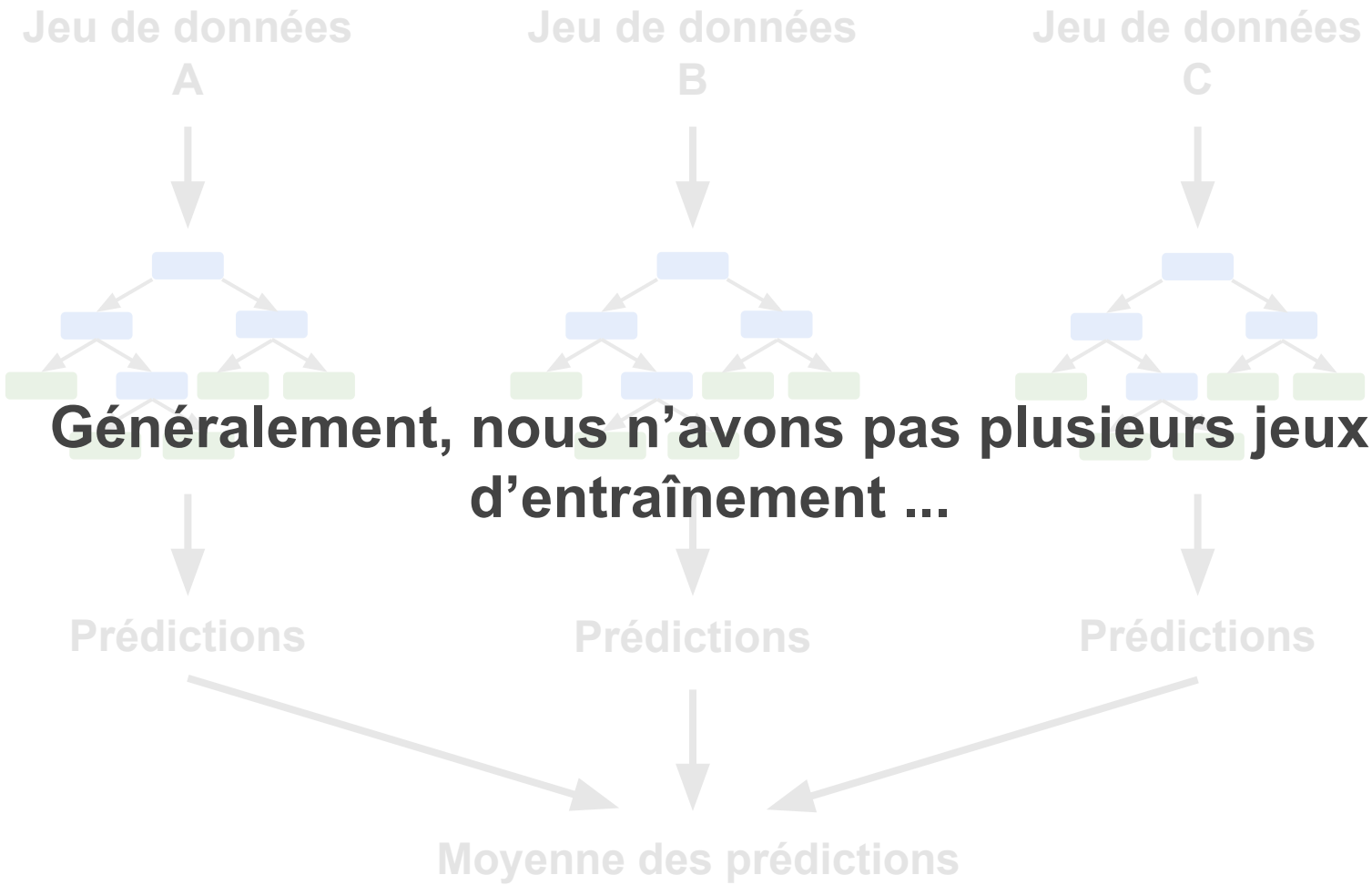
Quelques rappels de statistiques ...

Observations $x^{(i)}$	Variance des observations	Moyenne des observations	Variance de la moyenne
$x^{(1)}$	σ^2	$(x^{(1)}+...+x^{(m)}) / m$	σ^2 / m
$x^{(2)}$			
...			
$x^{(m)}$			

En d'autres mots, la moyenne d'un ensemble d'observations réduit la variance

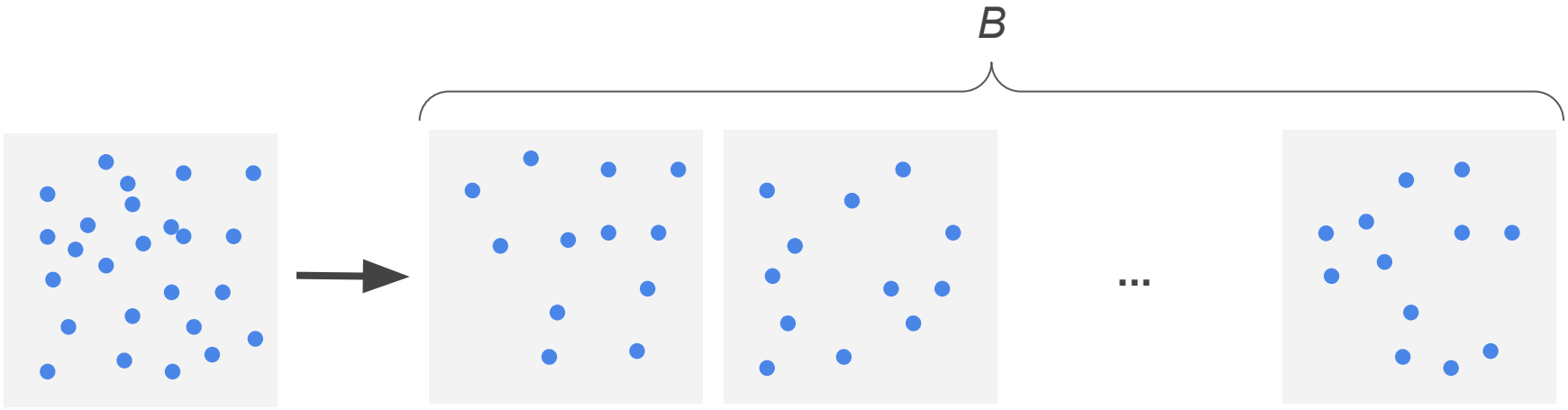
$$\sigma^2 > \sigma^2 / m$$
$$\forall\ m > 1$$


Jeu de données
A
Jeu de données
B
Jeu de données
C
Prédictions
Prédictions
Prédictions
Moyenne des prédictions
⇒ Réduction de la variance



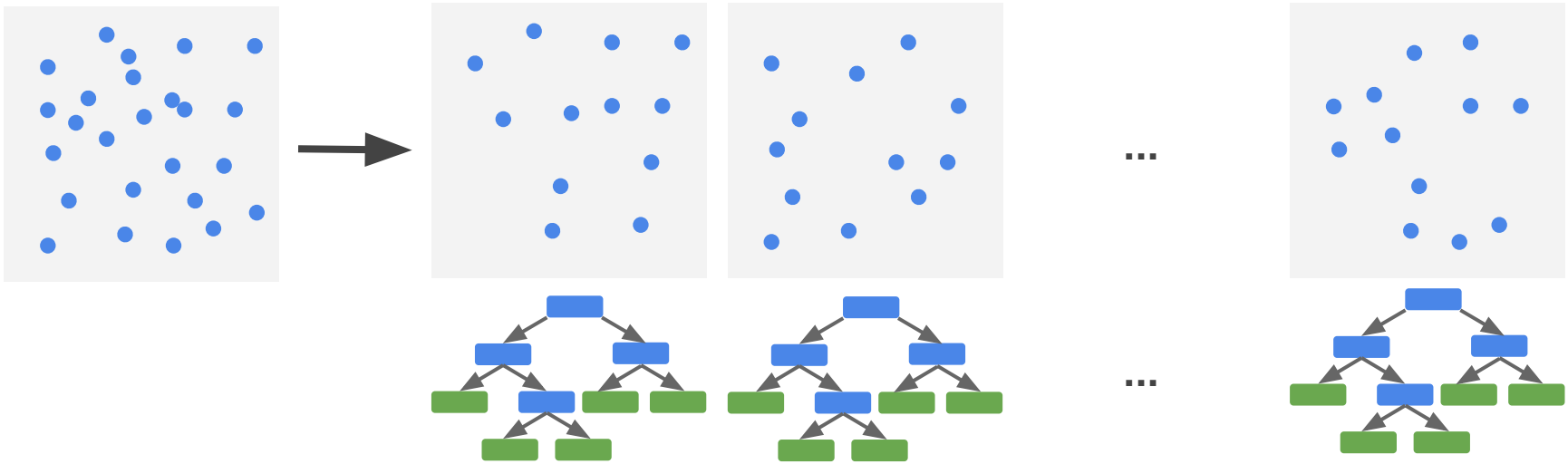
Généralement, nous n'avons pas plusieurs jeux d'entraînement ...

Bootstrap



La technique du **bootstrap** consiste à prendre des échantillons avec remise du jeu d'entraînement. Ci-dessus, *B* jeux de données "bootstrapés" sont créés

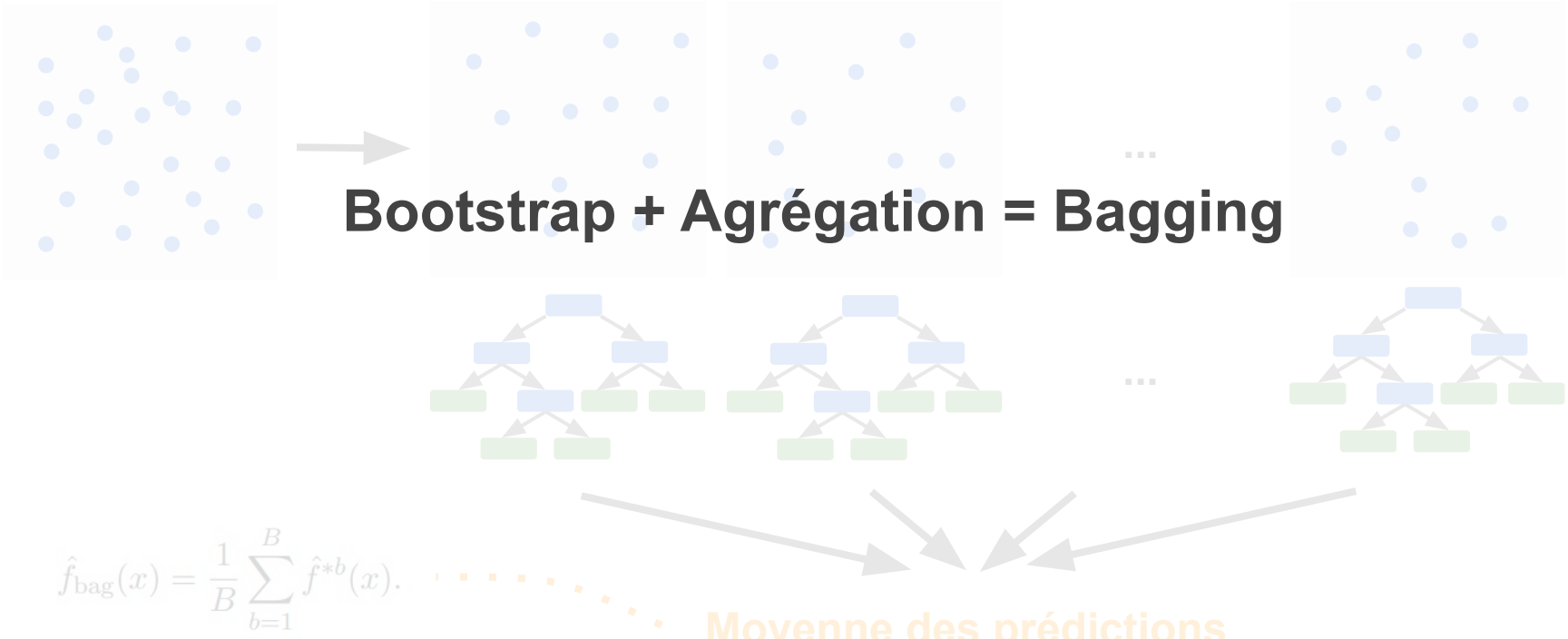
Agrégation



$$\hat{f}_{\text{bag}}(x) = \frac{1}{B}\sum_{b=1}^{B} \hat{f}^{*b}(x).$$

Moyenne des prédictions

Agrégation

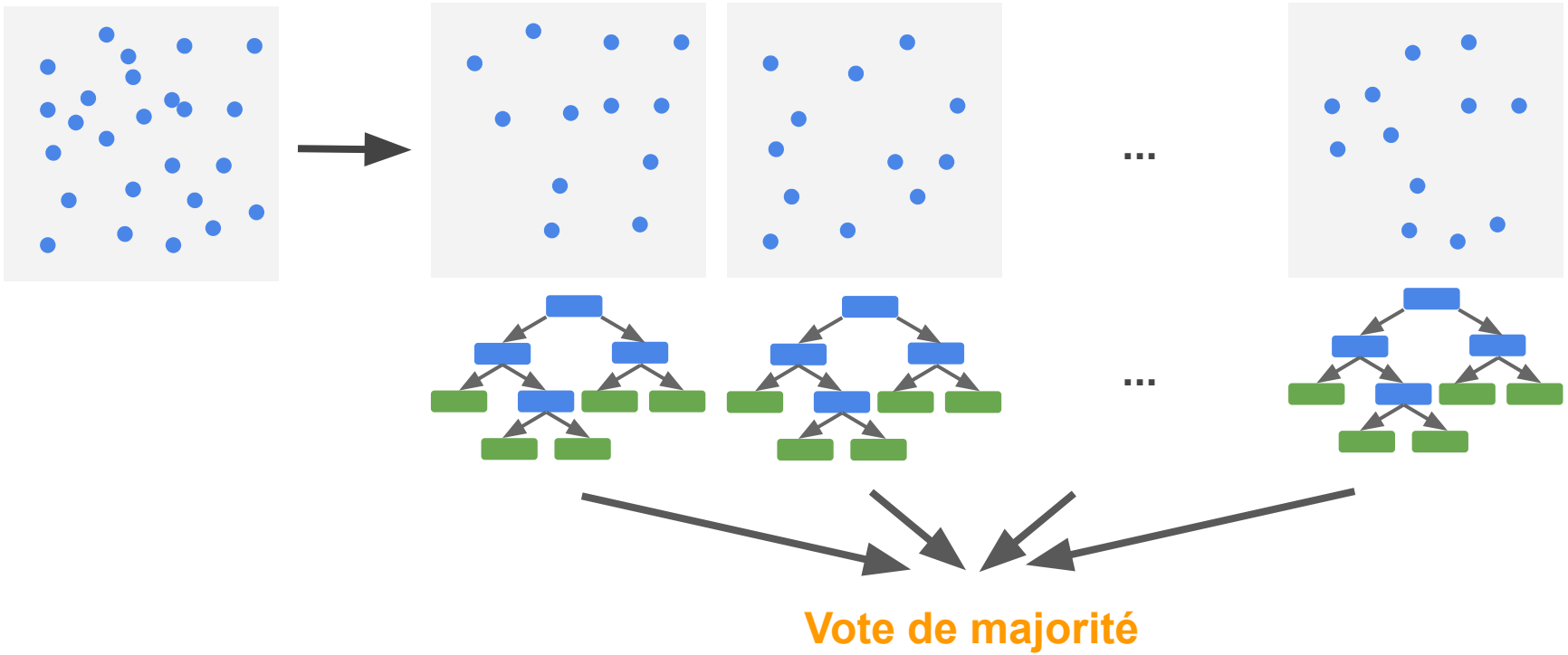


Bootstrap + Agrégation = Bagging

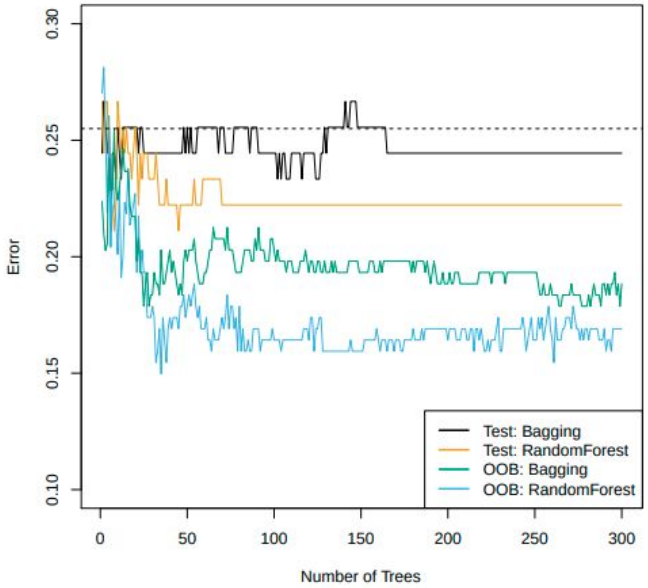
$$\hat{f}_{\text{bag}}(x) = \frac{1}{B}\sum_{b=1}^{B}\hat{f}^{*b}(x).$$

Moyenne des prédictions

Bagging des arbres de classification



Exemple
Jeu de données Heart



Source: Introduction to Statistical Learning

Un seul arbre de classification

Erreur Out-Of-Bag (OOB)

Erreur Out-Of-Bag (OOB)

L'**erreur OOB** est une méthode pour **estimer l'erreur de test**

Il peut être démontré que chaque arbre utilise en moyenne approximativement deux tiers des observations. **Le tier restant représente les observations OOB**

La prédiction pour une observation se base sur l'ensemble des arbres pour lesquels cette observation est OOB

Les *B*/3 (approx.) prédictions sont ensuite moyennées

Remarque: correspond au LOOCV lorsque B est large

BaggingClassifier (scikit-learn 0.22.2)

Estimateur

*class sklearn.ensemble.**BaggingClassifier**(**base_estimator=None**, **n_estimators=10**, max_samples=1.0, max_features=1.0, **bootstrap=True**, bootstrap_features=False, oob_score=False, warm_start=False, n_jobs=None, random_state=None, verbose=0)*

Nombre d'estimateurs

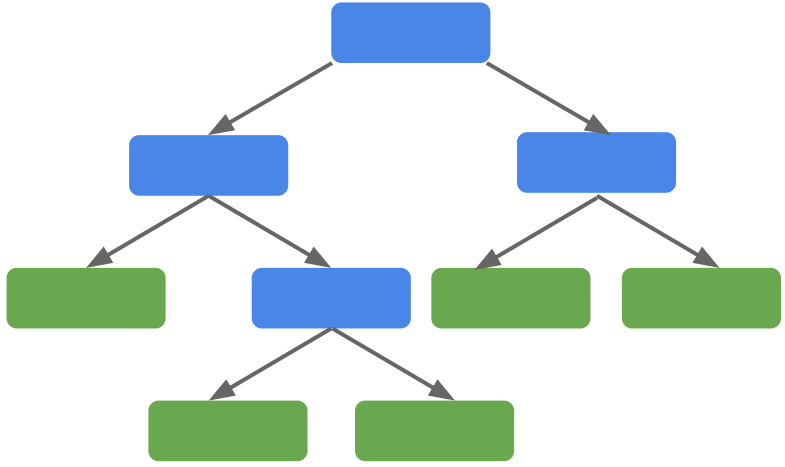
Si False, échantillonnage sans remise

En résumé ...

Le **bagging**, ou **agrégation bootstrap**, est une technique permettant de réduire la variance d'une méthode d'apprentissage supervisé

Très fréquemment utilisée dans le contexte des arbres de décision

Ne pas confondre **Bagging** et **Baggins** !

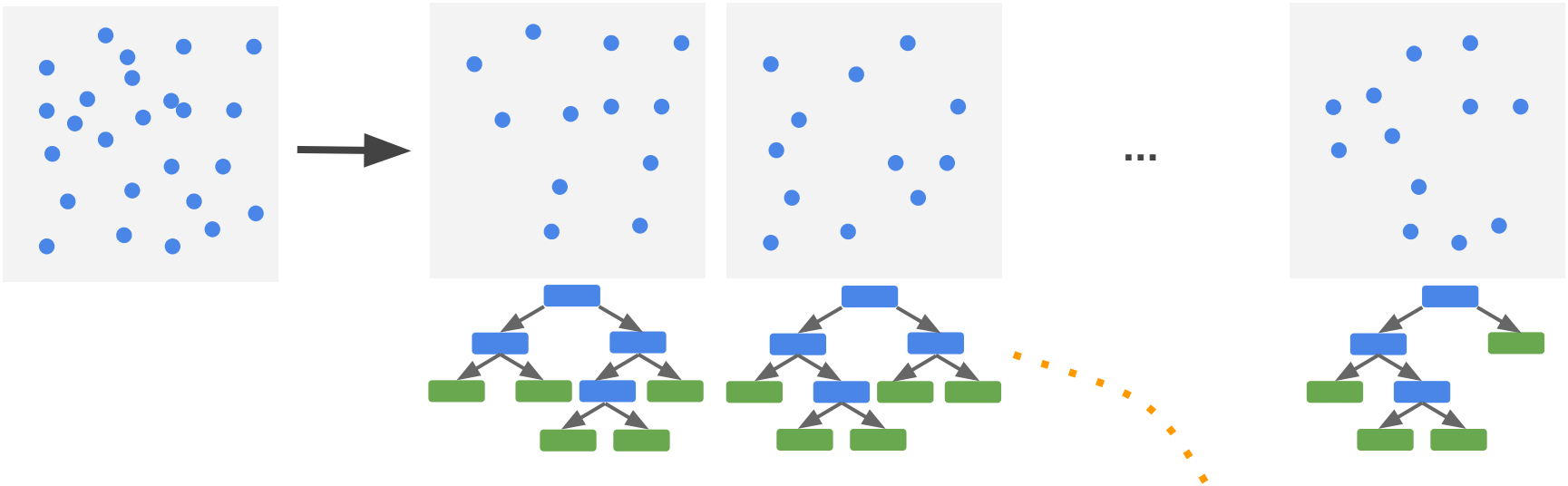




Sommaire

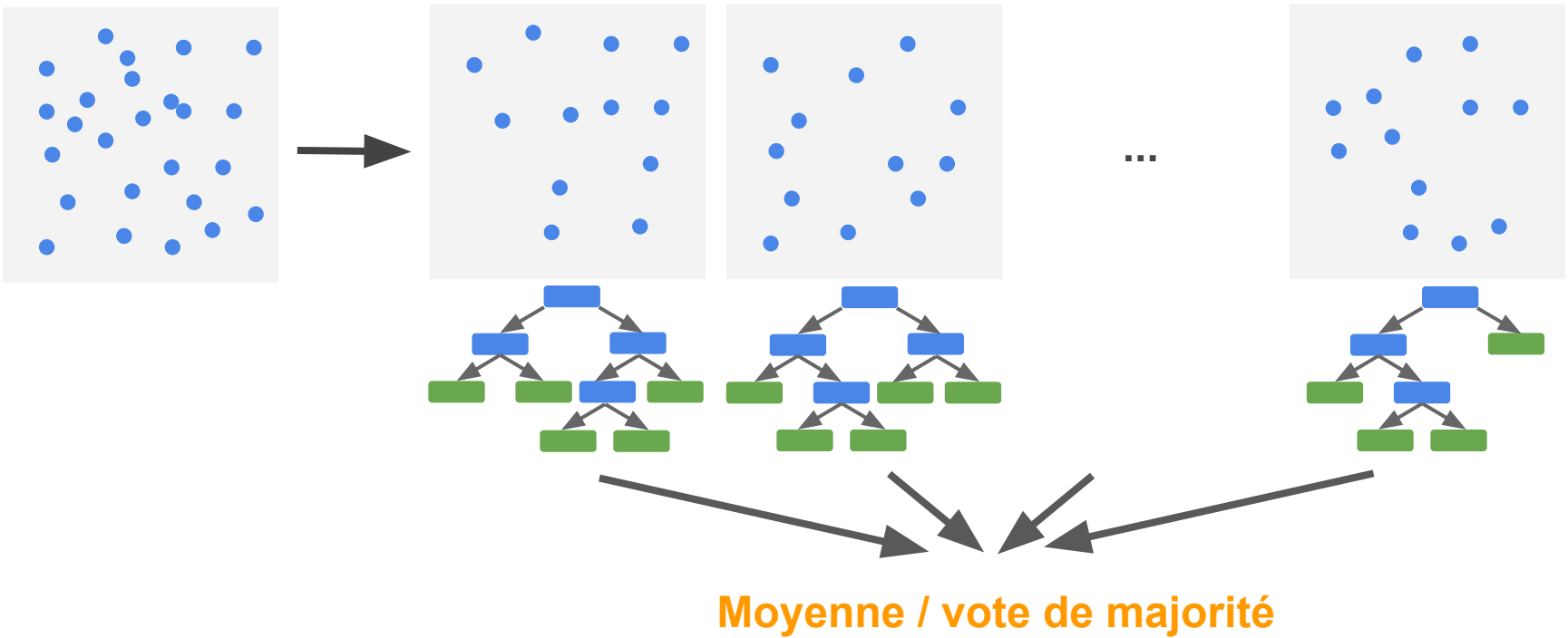
1. Introduction au Bagging
2. Forêts aléatoires
3. Introduction au Boosting
4. AdaBoost
5. Gradient Boosting
6. Importance des variables explicatives
7. Lectures et références

Forêts aléatoires / Random Forests

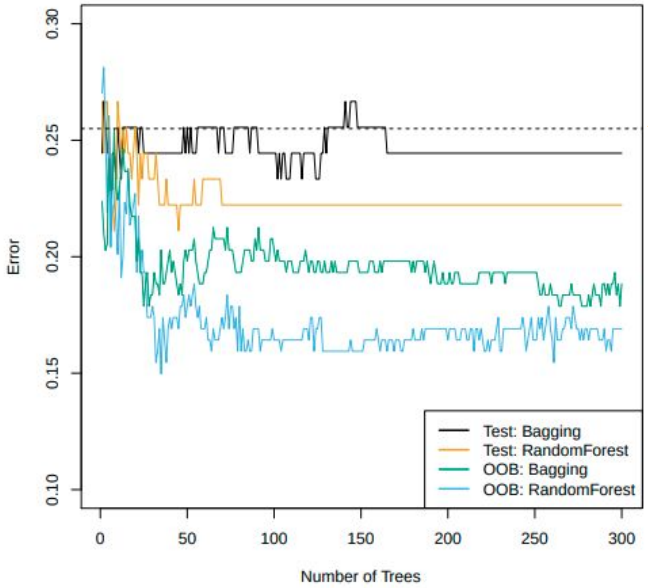


Les arbres sont **décorrélés** en ne considérant qu'un sous ensemble m des n variables explicatives à chaque partition. Typiquement racine carrée n

Forêts aléatoires / Random Forests



Exemple
Jeu de données Heart



Un seul arbre de classification

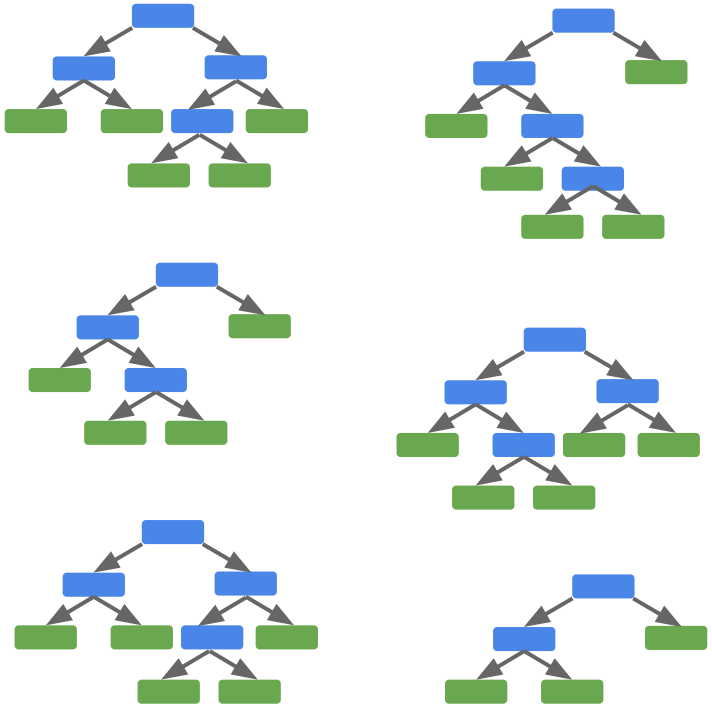
Erreur Out-Of-Bag (OOB)

Source: Introduction to Statistical Learning

Forêts aléatoires - hyperparamètres

Les forêts aléatoires **héritent des hyperparamètres des arbres de décision**

Il faut en plus déterminer le **nombre d'arbres composant la forêt** et choisir **le nombre de variables explicatives** à considérer à chaque étape de partition (méthode de sélection aléatoire)



RandomForestRegressor (scikit-learn 0.22.2)

Nombre d'arbres

*class sklearn.ensemble.**RandomForestRegressor**(**n_estimators=100**, **criterion='mse'**, **max_depth=None**, min_samples_split=2, **min_samples_leaf=1**, min_weight_fraction_leaf=0.0, **max_features='auto'**, max_leaf_nodes=None, min_impurity_decrease=0.0, min_impurity_split=None, **bootstrap=True**, oob_score=False, n_jobs=None, random_state=None, verbose=0, warm_start=False, **ccp_alpha=0.0**, max_samples=None)*

Nombre de prédicteurs à considérer à chaque partition

RandomForestClassifier (scikit-learn 0.22.2)

Nombre d'arbres

*class sklearn.ensemble.**RandomForestClassifier**(**n_estimators=100**, **criterion='gini'**, **max_depth=None**, min_samples_split=2, **min_samples_leaf=1**, min_weight_fraction_leaf=0.0, **max_features='auto'**, max_leaf_nodes=None, min_impurity_decrease=0.0, min_impurity_split=None, **bootstrap=True**, oob_score=False, n_jobs=None, random_state=None, verbose=0, warm_start=False, class_weight=None, **ccp_alpha=0.0**, max_samples=None)*

Nombre de prédicteurs à considérer à chaque partition

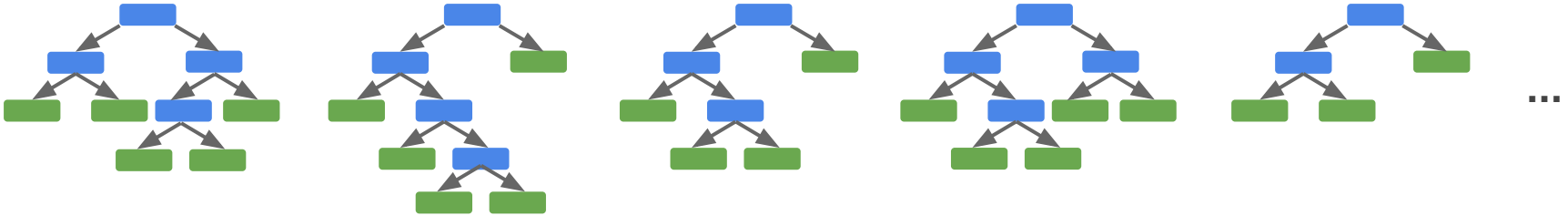
Sommaire

1. Introduction au Bagging
2. Forêts aléatoires
3. Introduction au Boosting
4. AdaBoost
5. Gradient Boosting
6. Importance des variables explicatives
7. Lectures et références

Comme pour le Bagging, le Boosting est une approche générale applicable à de nombreux algorithmes d'apprentissage supervisé en régression et classification

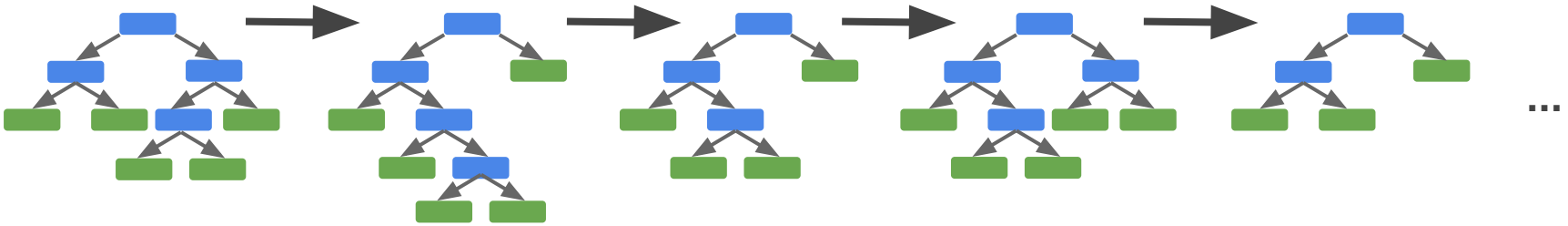
Nous allons voir ici le Boosting appliqué aux arbres de décision

Rappel



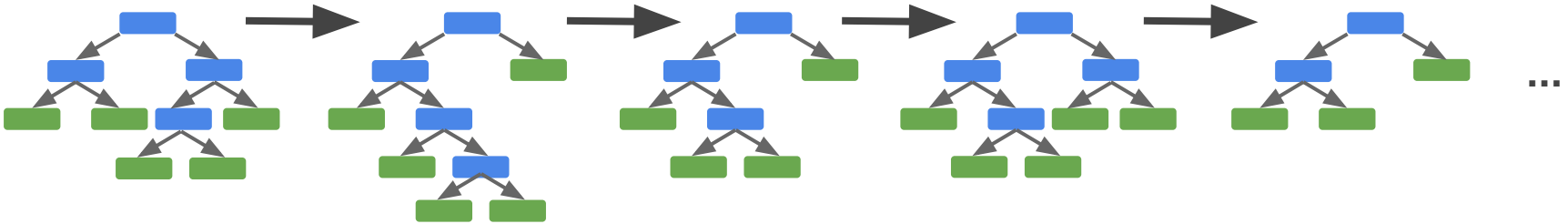
Dans les cas du Bagging et des forêts aléatoires, l**es arbres sont construits de manière indépendante**

Boosting



Le **Boosting** consiste à construire les arbres de **manière séquentielle**, chaque arbre étant construit à partir de l'information apprise par les arbres précédents

Boosting - Intuition



$x^{(i)}$	$p^{(i)}$
$x^{(1)}$	1.0
$x^{(2)}$	1.0

$x^{(i)}$	$p^{(i)}$
$x^{(1)}$	0.8
$x^{(2)}$	0.9

$x^{(i)}$	$p^{(i)}$
$x^{(1)}$	0.4
$x^{(2)}$	0.85

$x^{(i)}$	$p^{(i)}$
$x^{(1)}$	0.1
$x^{(2)}$	0.8

$x^{(i)}$	$p^{(i)}$
$x^{(1)}$	0.05
$x^{(2)}$	0.8

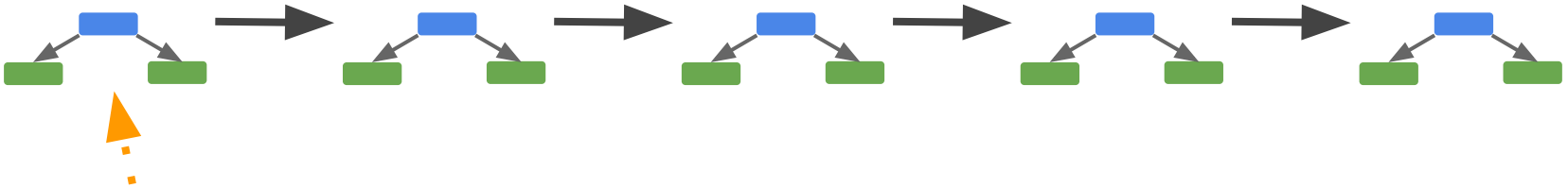
Les poids des observations sont ajustés en fonction de l'erreur de la prédiction précédente. Ainsi, les arbres ultérieurs se concentrent davantage sur les cas difficiles

Sommaire

1. Introduction au Bagging
2. Forêts aléatoires
3. Introduction au Boosting
4. AdaBoost
5. Gradient Boosting
6. Importance des variables explicatives
7. Lectures et références

AdaBoost

→ Apprenants faibles

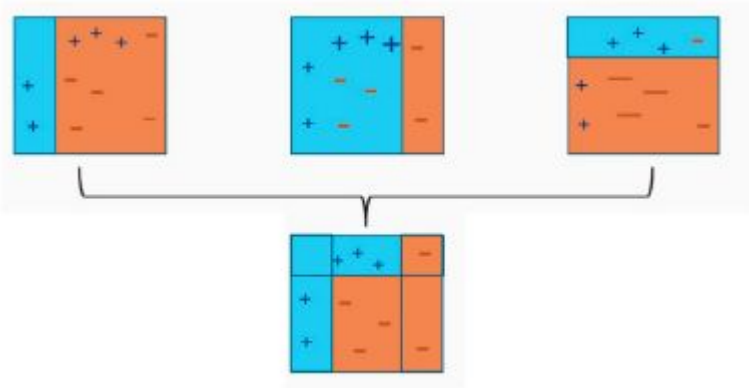


Une **souche** réalise une prédiction en utilisant qu'**une seule variable explicative**

→ Combinaison de plusieurs **apprenants faibles**

→ Capacité d'AdaBoost dépend du nombre d'apprenants (ici des arbres)

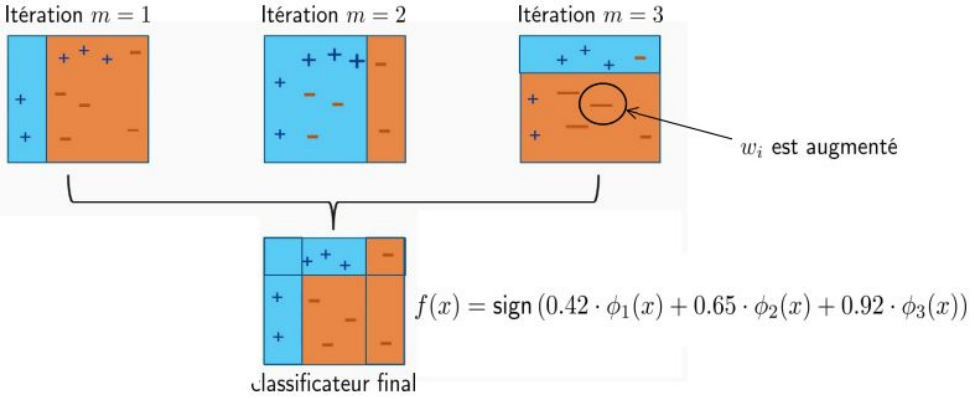
→ Les données ont un **poids** qui reflète leur difficulté à être classées



Algorithme d'AdaBoost (optionnel)

Algorithm 16.2: Adaboost.M1, for binary classification with exponential loss

1. $w_i = 1/N$;
2. **for** $m = 1 : M$ **do**
3. Fit a classifier $\phi_m(\mathbf{x})$ to the training set using weights $\mathbf{w}$;
4. Compute $\text{err}_m = \frac{\sum_{i=1}^{N} w_{i,m} \mathbb{I}(\tilde{y}_i \neq \phi_m(\mathbf{x}_i))}{\sum_{i=1}^{N} w_{i,m})}$;
5. Compute $\alpha_m = \log[(1 - \text{err}_m)/\text{err}_m]$;
6. Set $w_i \leftarrow w_i \exp[\alpha_m \mathbb{I}(\tilde{y}_i \neq \phi_m(\mathbf{x}_i))]$;
7. Return $f(\mathbf{x}) = \text{sgn}\left[\sum_{m=1}^{M} \alpha_m \phi_m(\mathbf{x})\right]$;



AdaBoostRegressor (scikit-learn 0.22.2)

Estimateur (ici souche)

*class sklearn.ensemble.**AdaBoostRegressor**(**base_estimator=None**, **n_estimators=50**, **learning_rate=1.0**, **loss='linear'**, random_state=None)*

Nombre d'estimateurs (ici # souches)

Pas

Fonction de perte

AdaBoostClassifier (scikit-learn 0.22.2)

Estimateur (ici souche)

*class sklearn.ensemble.**AdaBoostClassifier**(**base_estimator=None**, **n_estimators=50**, **learning_rate=1.0**, algorithm='SAMME.R', random_state=None)*

Nombre d'estimateurs (ici # souches)

Pas

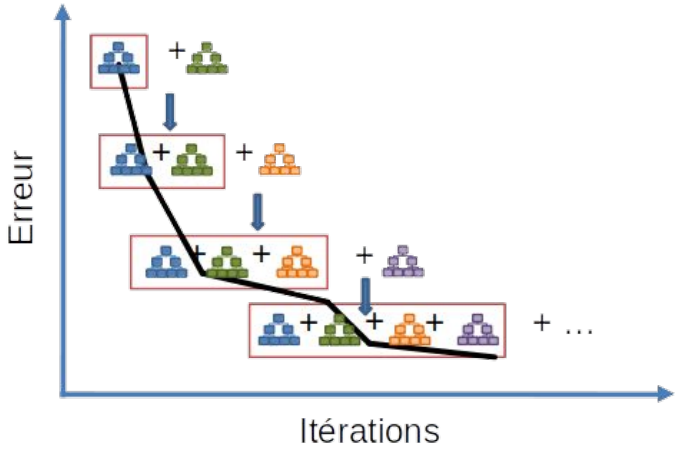
Sommaire

1. Introduction au Bagging
2. Forêts aléatoires
3. Introduction au Boosting
4. AdaBoost
5. Gradient Boosting
6. Importance des variables explicatives
7. Lectures et références

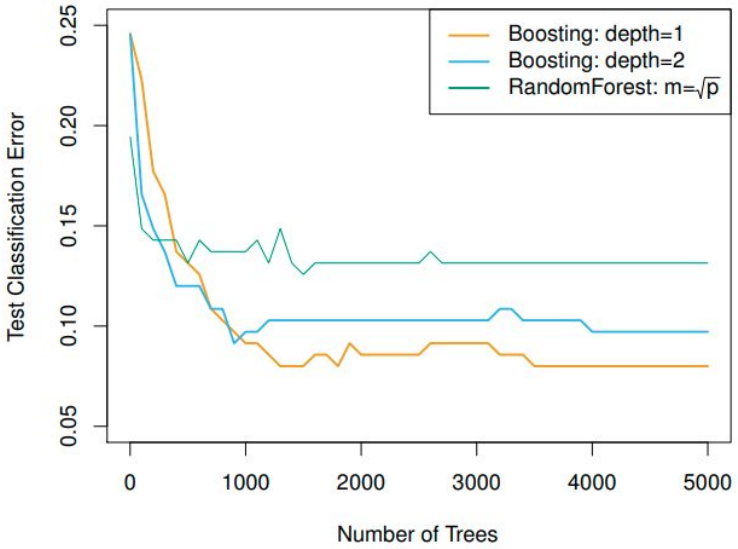
Gradient Boosting

Le **Gradient Boosting** est une généralisation d'AdaBoost) avec

- Une fonction de perte différentiable
- Un modèle de faible capacité
- Modèle additif



Exemple - Expression des gènes



Source: Introduction to Statistical Learning

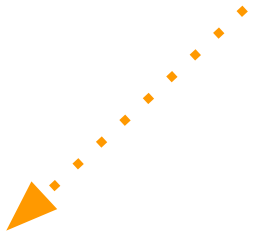
Prédiction de Cancer vs Normal

Le taux d'erreur de test pour un seul arbre est de 24%

Les arbres de profondeur 1 surpassent légèrement les arbres de profondeur 2 et surpassent tous les deux la forêt aléatoire

GradientBoostingRegressor (scikit-learn 0.22.2)

Fonction de perte



*class sklearn.ensemble.**GradientBoostingRegressor**(**loss='ls'**, learning_rate=0.1, n_estimators=100, subsample=1.0, criterion='friedman_mse', min_samples_split=2, min_samples_leaf=1, min_weight_fraction_leaf=0.0, max_depth=3, min_impurity_decrease=0.0, min_impurity_split=None, init=None, random_state=None, max_features=None, alpha=0.9, verbose=0, max_leaf_nodes=None, warm_start=False, presort='deprecated', validation_fraction=0.1, n_iter_no_change=None, tol=0.0001, ccp_alpha=0.0)*

GradientBoostingClassifier (scikit-learn 0.22.2)

Fonction de perte



*class sklearn.ensemble.**GradientBoostingClassifier**(**loss='deviance'**,*
learning_rate=0.1, n_estimators=100, subsample=1.0, criterion='friedman_mse',
min_samples_split=2, min_samples_leaf=1, min_weight_fraction_leaf=0.0,
max_depth=3, min_impurity_decrease=0.0, min_impurity_split=None, init=None,
random_state=None, max_features=None, verbose=0, max_leaf_nodes=None,
warm_start=False, presort='deprecated', validation_fraction=0.1,
n_iter_no_change=None, tol=0.0001, ccp_alpha=0.0

XGBoost

XGBoost est une bibliothèque **open-source** de gradient boosting pour C++, Java, Python, R, Julia, Perl, et Scala

Using XGBoost in Python ← **À faire !**

XGBoost Documentation



LightGBM

LightGBM = Light Gradient Boosting Machine

Libraire **Open-source** créé et maintenue par **Microsoft**



LightGBM's documentation

LightGBM, Light Gradient Boosting Machine

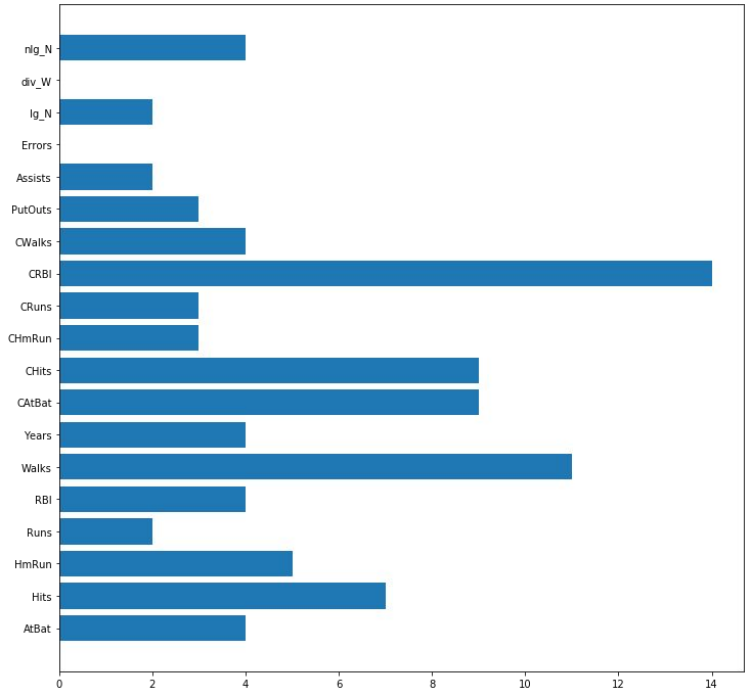
Sommaire

1. Introduction au Bagging
2. Forêts aléatoires
3. Introduction au Boosting
4. AdaBoost
5. Gradient Boosting
6. Importance des variables explicatives
7. Lectures et références

Importance des variables explicatives

Pour les **arbres de régression**, on accumule la diminution du RSS attribuée à un prédicteur donné

Une grande valeur moyennée sur les B arbres signifie donc que la variable explicative est importante

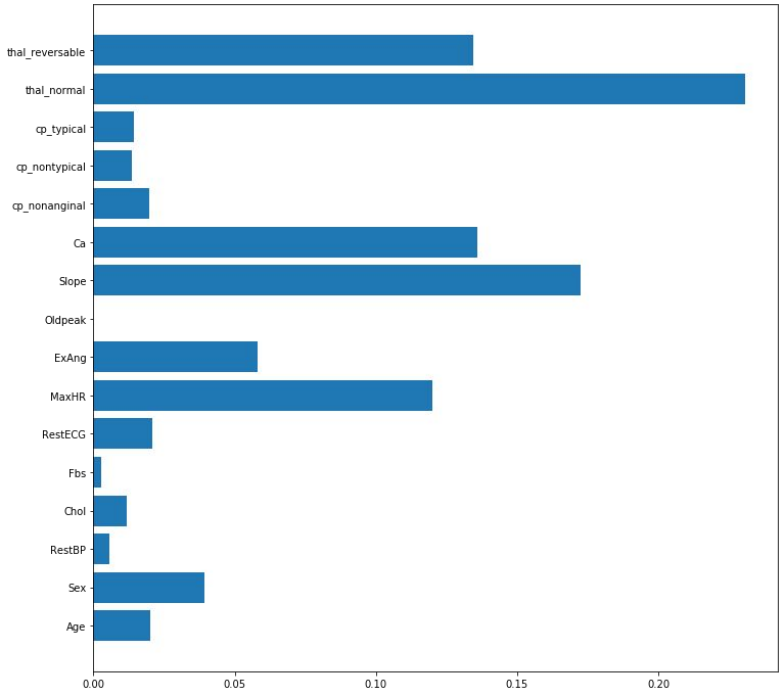


Importance des prédicteurs (Hitters)

Importance des variables explicatives

Pour les **arbres de classification**, on accumule la diminution de l'index de Gini attribué à un prédicteur donné

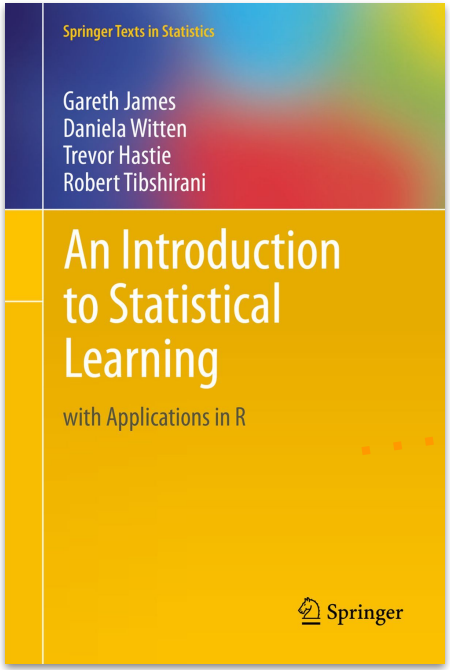
Une grande valeur moyennées sur les B arbres signifie donc que la variable explicative est importante



Importance des prédicteurs (Heart)

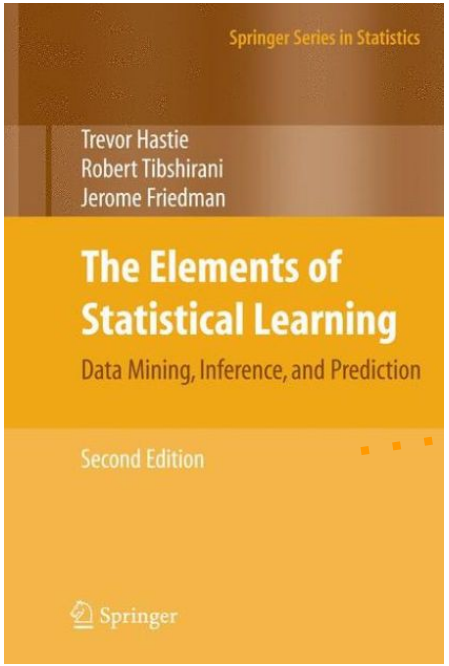
Sommaire

1. Introduction au Bagging
2. Forêts aléatoires
3. Introduction au Boosting
4. AdaBoost
5. Gradient Boosting
6. Importance des variables explicatives
7. Lectures et références



8 Tree-Based methods
p. 303-336

Gareth James, Daniela Witten, Trevor Hastie and Robert Tibshirani, **"An Introduction to Statistical Learning with Applications in R"**



10 Boosting and Additive Trees
p. 337-388

Trevor Hastie, Robert Tibshirani and Jerome Friedman, **“The Elements of Statistical Learning”**

Lectures et références

[1] Gareth James, Daniela Witten, Trevor Hastie and Robert Tibshirani**, “An Introduction to Statistical Learning with Applications in R”**

[2] Cours “Classification et métriques d’évaluation”, Camille Besse, CRDM